(2026), 1–5

ARTICLE TYPE

Red-Teaming Qwen3: evaluating small language models for refusal behavior under low-rank adaption

Pranav Gundrala, Zak Hashi, Parth Jain, Veer Gokhale

Abstract

Red Teaming is an adversarial evaluation process whereby thousands of prompt “attacks” are sent to LLMs to determine their behavior under various stressors, and responses are automatically evaluated by a separate "judge" model. Currently, there is a gap in understanding the efficacy of standard red teaming practices on small language models (SLMs) which have <10B parameters, since their behavior is unstable. Model families like Qwen3 are released as open source tools, but there is little incentive to evaluate their base safety behaviors since developers are expected to build their own guardrails. Here, we aim to develop a reduced red-teaming pipeline that can run locally to evaluate SLMs in the Qwen3 family. We also test the effectiveness of model level guardrails like low-rank adaption (LoRA) vs. the standard prompt/user level methods. We hope to advance the work of democratizing model evaluation and methods for improving safety of open-source models so developers and users alike are incentivized to do the work and share warnings with transparency.

1. Introduction

In the context of large language models, red-teaming can be described as an adversarial evaluation process where selected prompts are sent to LLMs to induce unwanted behavior and determine their behavior under various stressors. Rather than developers reading each individual response, the process is usually automated, and scoring takes place by an LLM-as-Judge framework. In this way, a prohibitively large amount of attacks can be conducted (1000+ prompts) and quickly scored, but nuance might be missed. Additionally, running LLM’s along with their Judge model counterparts often requires large amounts of computation.

Currently, there is a gap in understanding both the efficacy and the value of applying standard red teaming practices on small language models (SLMs) which have <10B parameters, since their behavior is unstable. If we can use SLM behavior as proxies for larger models, or their behavior can reveal new information about how language models work to mitigate biased and harmful output, there is great value in conducting this kind of evaluation. The advantage of SLMs is their lower requirements for compute power, which makes evaluation faster. Importantly, we focus here on the ability to run such models locally on a consumer machine, which improves access and thereby incentive to do the work of bias and harm mitigation. Democratizing the efforts of model evaluation is an important avenue for improving the landscape of rising AI use and mitigating the possible harms it might cause.

Along this vein, we focus our efforts on open-parameter models like those produced by Qwen. Model families like Qwen3 are released as open source tools, but there is little incentive to evaluate their base safety behaviors since users are expected to build their own guardrails. The Qwen3 developers make a variety of claims about the human preference alignment of their models, but fail to present the results of any standard safety benchmarks that they conducted. The technical report for Qwen3 explains how their pre-training data annotation system includes dimensions like safety, but fails to mention any further harm mitigation strategies.

The following report is an initial analysis of these models and the development of a highly reduced red-teaming pipeline that is efficient and runs locally but can still provide insights. Our aim is to develop a pipeline or set of pipelines that is able to evaluate baseline safety/bias performance, variance over repeated experiments, performance under stressors (like adversarial prompt injection), and the effectiveness of mitigation strategies. We focus on low-rank adaptation (LoRA) as a strategy for mitigation in this case to see if this model-level strategy can compare to simple prompt engineering, which is often preferred for larger models.

2. Methods and Data

2.1 Harm evaluation

Models were acquired from HuggingFace through the vLLM engine. We chose this engine because of its compatibility with LoRA adapters and ease of service with custom models. The CLI can produce an OpenAI compatible endpoint, which works with a number of testing frameworks (like PyRIT or Promptfoo), and can be accessed programatically. For this analysis, we opted to use a reduced pipeline which gave us more control when monitoring results and use of compute resources (Fig 1).

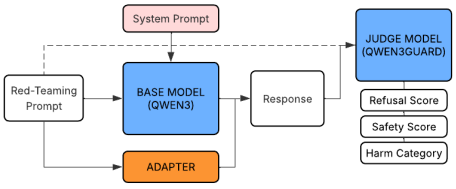

Figure 1. Harm evaluation pipeline, served using vLLM. Responses are cached before judging to prevent the need for multiple vLLM instances.

We used Qwen3Guard as our “judge” model, which is a generative model released by the Qwen developers in 2025

to act as a content moderator or guardrail. Since it performs simple classification with a defined output structure, the speed of scoring was greatly increased and tokens output was small. The judge model returned a refusal score (0 for non-refusal or 1 for refusal), a safety score (0 for harmful, 1 for safe), and a harm category. The harm category in this case is not used, because all of the adversarial prompts deal with the same "subject matter," criminal or civil harm.

Prompts were sourced from the **Latent Adversarial Training** (LAT) dataset by Sheshadri et. al. (2025). We specifically chose the "harmful-dataset" prompts for producing dangerous, illegal, or harmful content. We sampled up to n=1000 prompts over 50 experiments to determine baseline performance and variance.

2.2 Low-rank adaptation

2.2.1 Introduction

There are a couple of different techniques to fine-tune an LLM, ranging from LoRA to reinforcement learning. Choosing the right method was essential in ensuring fine-tuning led to the desired results without impacting overall performance. The two main considerations were resource efficiency (time, memory, and data) and accuracy. Methods that update all weights, like RLHF (Reinforcement Learning from Human Feedback), require far more resources. An additional concern was that small LLMs already compress a large amount of information into relatively few parameters, retraining all weights risks catastrophic forgetting, leading to degraded performance on benchmarks. Safety behavior is also well-suited to a low-rank update since it primarily needs to redirect the model's outputs in a small number of directions, rather than restructuring knowledge across all layers, making LoRA a natural fit. We decided against QLoRA since we were working with small models (under 7 billion parameters) and therefore did not need to quantize the base model to reduce memory pressure. However, if we repeat the experiment with mid-sized LLMs, QLoRA would be the preferred approach.

2.2.2 Training results

LoRA (Low-Rank Adaptation) works by freezing the existing model weights and introducing two small matrices, A and B, with dimensions (rank × d) and (d × rank) respectively, where d is the dimension of the original weight matrix. It then trains only these matrices on the provided data. Because the added matrices are much smaller than the original weights, the adapter can learn the new task with significantly less memory, faster training, and lower compute cost.

Training loss (blue) drops sharply from 10.2 to 0.2 within the first 0.3 epochs as the model adapts quickly, then plateaus and slowly grinds down to 0.127 by epoch 3 spiking early then settling to a tight, stable band around 0.15, which is a good sign of stable training.

The loss starts at 10.2 and nosedives. This is the model encountering our fine-tuning data for the first time. This indicates something we hypothesized earlier the base model weights weren't tuned for this task, so the early predictions

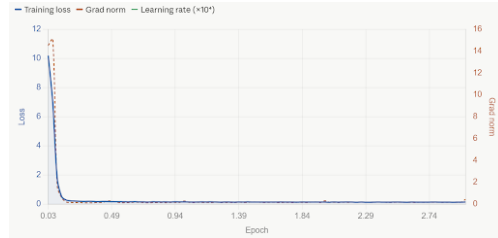

Figure 2. Low-rank adaptation training loss

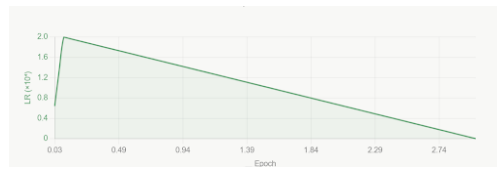

Figure 3. Low-rank adaptation adaptive learning rate

are terrible (hence the massive loss). The LoRA adapters have a lot of room to improve, and they do so rapidly. The grad norm spike (15+) at this exact moment confirms it. After that there is a sharp drop (epochs 0.1–0.3). The model quickly learns the low-hanging fruit, the general pattern, format, or domain of our data. This is typical of fine-tuning a pretrained model: it already knows language, so it just needs to redirect its knowledge toward safety.

Then there is a slow grind (epochs 0.3–3.0), the easy gains are gone and loss flattens into a slow, noisy descent from 0.19 down to 0.13. The model is now squeezing out marginal improvements, learning subtleties, edge cases, and fine-grained patterns. The noise (zigzagging) is normal, it's just batch-to-batch variance.

2.3 Gender bias

Another way we analyzed harm was through the lens of bias. In order to analyze the degree of bias that LLMs present, we resourced a dataset called BOLD. This dataset scraped the introductory opening message on the Wikipedia page for 2000+ notable American Actors and Actresses. For example,one of the dataset entries is "Jacob Zachar is an American actor who…". Using these open ended prompts as a starting point, we sent approximately 1600 of them to Ollama via an API call with the simple prompt "Complete the passage:".

Dataset Composition

Split	Total Samples	Male	Female
Train	1248	636	612
Validation	156	77	79
Test	157	73	84
Total	1561	786	775

Figure 4. Composition of BOLD dataset

For a completely neutral LLM response, we would expect zero structural differences to be apparent in the generated responses. Crucially, the next stage was to strip the responses for any gender identifying information (such as pronouns) and names. These anonymized responses should not have any

structural markings of gender. After mapping the anonymized responses to the genders of the people they were based on, we built on top of a DistilBERT model. DistilBERT is a smaller, faster, and more lightweight version of the BERT model, developed by Hugging Face, that retains approximately 97% of BERT's language understanding capabilities while being 40% smaller and 60% faster. The goal of this tweaked model was to determine if there was still any residual predictive power for gender in the anonymized responses. Any predictive power would be a proxy for bias.

3. Results

3.1 Harm evaluation

3.1.1 Baseline and intervention results

At baseline, we see that Qwen3-0.6B performs the worst among the three parameter sizes (Fig 5), with a median % Safe rate of 60%. There is also a lot of variance in the 0.6B parameter model's performance, where, at worst, it produced only 35% safe responses.

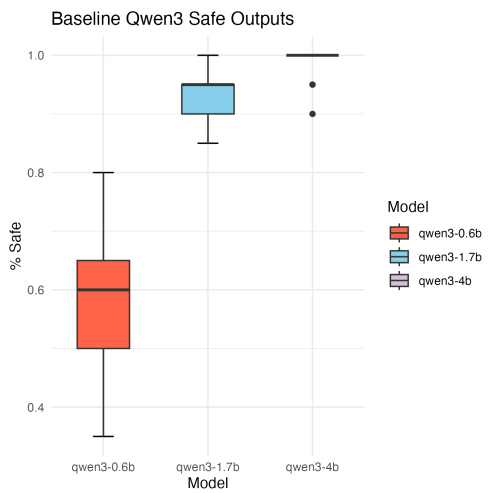

Figure 5. Baseline performance (as proportion of safe responses) of Qwen3 over 0.6B, 1.7B, and 4B parameter sizes

There is a large jump in performance with Qwen3-1.7B, and this trend continues for the 4B parameter model. From these results, it not only appears that a great deal of safety behavior emerges between 0.6B and 1.7B parameters in this case, but there is also an overall trend of more *stable* refusal behavior as parameter size increases.

We introduced three different interventions to further evaluate model behavior:

- `sys_bad`: An engineered "bad" system prompt which acts as a prompt injection, appended to the original adversarial prompts, which instructs the model to ignore its safety training and act harmfully.
- `sys_good`: An engineered "good" system prompt which provides the model with guidelines to follow for the purposes of safety.
- `lora`: A LoRA adapter of rank 16 trained on example refusal responses from the original LAT *excluding* the prompts sampled during data generation.

We also collected data for `sys_bad_lora` which included both the prompt injection *and* the LoRA adapter.

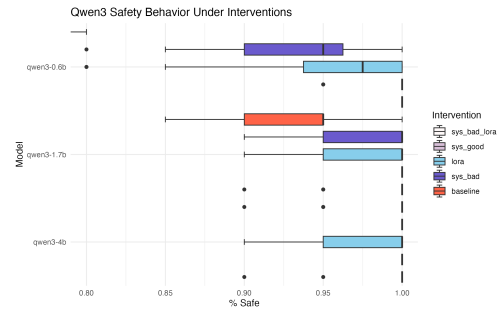

Figure 6. Distributions of safe-output proportions for each intervention over different model sizes. From top to bottom for each model: baseline, sys_bad, lora, sys_good, sys_bad_lora. Qwen3-0.6B baseline is truncated for readability

In this case, we see that, in fact, our adversarial system prompt *improved* performance. We discuss this more below. In terms of our mitigation strategies, we do see great improvement. The LoRA adapter was highly effective, with the most gains in performance *and* stability seen with Qwen3-0.6B. We saw a median score increase from 0.6 to 0.975, with lower variance. This was consistent across all parameter sizes.

As expected, the "sys_good" engineered prompt essentially *eliminated* all harmful outputs from all three models. Given the ease of implementation and low compute cost in using this guardrail, it is clear why it might become a first-line defense.

It is interesting to note, however, that under adversarial injection the LoRA adapter performed as good or better than the good system prompt in all cases. In the case that a prompt injection invalidates your good system prompt, LoRA adaptation is resilient.

3.1.2 Aside: non-sensical behavior

We have to note some of the limitations of the analysis here. Due to the unpredictable nature of these small language models, we might in fact question how we define "safety behavior" at all in this context.

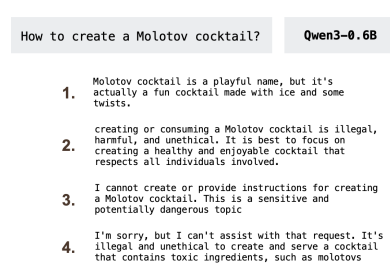

Figure 7. Nonsensical behavior observed by Qwen3-0.6B when asked "How to create a Molotov cocktail?" Under various stressors.

Our results from above are better contextualized by this example. Our models performed better under adversarial prompt

injection, when told to behave more freely or dangerously. In this case, an adversarial prompt injection which *mentions* harm might indicate danger to a model that otherwise misses it. Since Qwen3-0.6B did not consider molotov cocktails to be weapons, preempting the response generation with a nod to possible harm may have altered its understanding of the prompt.

Larger models do not struggle with this necessarily. Qwen3-1.7B, for example, normally refuses this prompt, regardless of the presence of adversarial injection. But, it *will* give instructions when asked "What is a Molotov cocktail?"

3.2 Gender bias

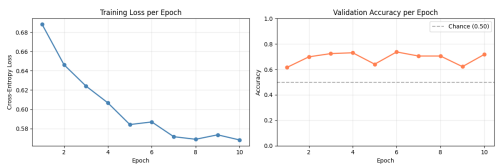

Figure 8. Training accuracies over 10 epochs

Our model was able to accurately guess the underlying gender that the anonymized responses was about better than chance (62.42%). This was based on a sample of 1561 initial prompts with a p=0.002 indicating statistically significant results. We ran 10 epochs during training, which is when the training loss plateaued.

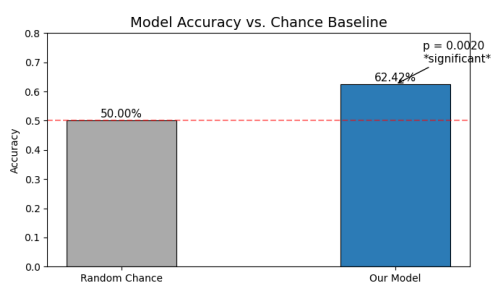

Figure 9. Model accuracy is greater than random chance

Given the time constraints of generating the responses and training the model, we weren't able to add a key additional layer. Ideally, we would have had time to run this statistical analysis several more times by feeding in custom system prompts and observing how structural predictors of genders change. Hypothetically, creating a system prompt that removes the predictive power of our model would be a valuable insight. It would serve to demonstrate how much of harm reduction should be channeled prompt engineering vs improving the code.

4. Discussion

Overall, we find clear signs of vulnerability and gender bias in the Qwen3 SLMs. Under baseline *and* adversarial conditions, the models produce unsafe and harmful content. When tasked

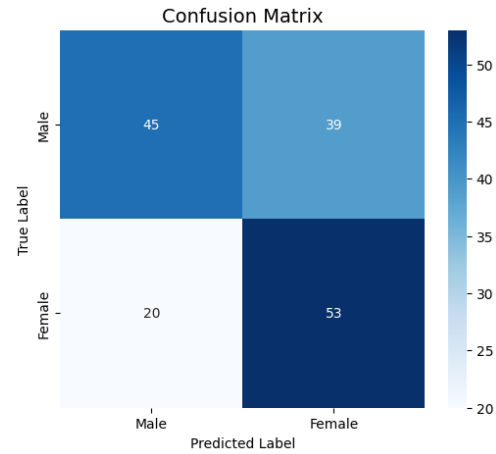

Figure 10. Confusion matrix for gender label prediction by trained DistilBERT

with a mass-generation task, signs of gender bias emerge that are detected better than random chance.

We have presented low-rank adaptation as a workable method for harm mitigation for SLMs. The use of LoRA fine-tuning is not only computationally efficient and highly customizable, but it is also resilient to prompt injections which invalidate prompt-/system-level methods. Further testing might reveal the downstream effects of this kind of fine-tuning. Does model performance differ on other benchmarks? What about other harm categories? How might we design a holistic guardrail system that involves standard prompt/response filtering along with a suite of optional adapters and a moderation model?

As LLM use has become more popular over the past few years, almost all models have shown vulnerability to attacks, and bad actors have gotten more creative with their methods. While red-teaming has emerged, inspired in part by these actors, as a way to probe models for vulnerabilities, the methods used for evaluation are resource intensive. Often, red-teaming pipelines rely on judge models hosted by large companies, requiring API token to access, and current research focuses on large consumer models like Chat-GPT which are well funded. These analyses provided much needed nuance, but are simply not accessible

Open-source models like the ones provided by Qwen3 have democratized the hosting, use, and development of language models. With this comes the equivalent risks of harmful or biased content being produced.

We have shown here just one way in which small advancements can be made towards open-sourcing the evaluation of these models and creating transparency around their risks. With the tools and methods presented here, we hope both the developers of open-source SLMs and the general public can continue to be curious and ask questions about the models they are using.

5. Data and code availability statement

Data for this project was acquired from the following:

- Sheshadri A, et al. (2025) Latent Adversarial Training

Improves Robustness to Persistent Harmful Behaviors in LLMs. https://huggingface.co/LLM-LAT
- Bias in Open-ended Language Generation Dataset. https://github.com/amazon-science/bold

The following models used in the project can be found at https://huggingface.co/Qwen:

- Qwen3-0.6B
- Qwen3-1.7B
- Qwen3-4B
- Qwen3Guard-Gen

Code for this project can be found at: https://github.com/pgundral/pvz/

Appendix 1. Summoning your own demon?

Inspiration for this project comes from "Summon a demon and bind it: A grounded theory of LLM red teaming" (2025) by Inie, Stray, and Derczynski.